Professional development for information security specialists

PRACTICAL WORK GUIDE
FOR STUDENTS

course:
“Methods and Principles of Designing Modern Secure Systems. Using Artificial Intelligence Methods for Data Analysis in Information Security Systems”

PRACTICE 3. DETECTING ATTACKS IN AN EVENT LOG

Lab setup, writing rules, detection

(2 academic hours)

2026

Contents

Goal and plan 3

Step 1. Create a virtual machine 4

Step 2. Which logs the system keeps 5

Step 3. Docker and the vulnerable application 6

Step 4. A step-by-step attack in the browser 7

4.1. Password guessing 7

4.2. Password spraying 7

4.3. Looking for other people’s orders 8

4.4. Switching off MFA 8

Step 5. The log and the first detection rule 9

Step 6. The attack and its detection 10

Base rules 10

Password guessing 10

Password spraying 10

Successful login after many failures 11

Prepare the control log 11

Apply all rules 11

Step 7. Tuning and the report 13

Extra task: catch an IDOR 14

If something went wrong 15

Goal and plan

This practice takes you through the whole way from a clean system to the detection of an attack. You create a virtual machine with Ubuntu, study its logs, start a vulnerable web application in Docker, explore the application by hand in a browser, write detection rules in the Sigma format, run an attack and detect it, and tune a rule to reduce false alarms.

The result is a set of detection rules, the alerts they produce, and a proposal to reduce false alarms. You write them in a report that follows the template `report-template.md`.

Each step ends with an **expected result**. Check it before you go to the next step.

You need a hypervisor (VirtualBox or similar), the installation image of Ubuntu Server 24.04 and the folder with the practice materials. The teacher gives you these. On the virtual machine you use Docker (you install it in step 3) and Python 3 (it is part of Ubuntu). The commands for the terminal and the content of the rule files are given in full in the text.

Step 1. Create a virtual machine

You can start the installation and read the next step while it runs.

1. In the hypervisor, create a virtual machine: type Linux, version Ubuntu (64-bit), memory 2048 MB, disk 20 GB.
2. Connect the installation image of Ubuntu Server to the virtual drive and start the machine.
3. Go through the installer: default keyboard layout, install on the whole disk, user name and password of your choice (remember them). On the screen “SSH Setup” turn on **Install OpenSSH server**.
4. Wait until the installation ends, restart the machine and log in with your account.

Check the version of the system:

```
lsb_release -d
```

Expected result: a line like `Description: Ubuntu 24.04 LTS`. You are logged in and you see the command prompt.

Step 2. Which logs the system keeps

Ubuntu records security events even before you install any application. Let us look at them.

The latest login events:

```
journalctl -n 20
sudo cat /var/log/auth.log | tail -n 20
```

Create a security event by hand: a failed `sudo` with a wrong password.

```
sudo -k            # forget the saved password
sudo whoami        # type a WRONG password once, then the right one
```

Find this failure in the log:

```
sudo grep "authentication failure" /var/log/auth.log | tail -n 3
```

Expected result: the output has a line with `authentication failure` and your user name. This is a security event that the operating system recorded.

Question for the report (item 1): which field of the line shows the terminal or the address that the failed login came from?

Step 3. Docker and the vulnerable application

Install Docker with the official script:

```
curl -fsSL https://get.docker.com | sudo sh
sudo usermod -aG docker $USER && newgrp docker
docker --version
```

If the room has no internet access, the teacher gives you a ready package or image. Follow the teacher's instructions.

Start the training application "Shop" from the folder with the practice materials:

```
cd app
docker compose up -d --build
docker compose ps
```

Open `http://localhost:8080` in the browser. You see the interface **TraceShop LAB**. On the left is the console of the researcher, on the right is the live telemetry, and on top are four missions. If you only need a technical check, open `http://localhost:8080/health`.

The interface of the application is in Russian. In this guide the buttons are given in English, and the Russian label is in brackets.

Press the button **Reset the lab** ("Сбросить стенд") to clear the events of earlier runs.

Expected result: `docker compose ps` shows the container `shop-app` in the state `Up`, and the browser opens TraceShop LAB with the counter `0 событий` (0 events).

Step 4. A step-by-step attack in the browser

The application writes every important action to `app/logs/events.jsonl`. The missions on the site must be done strictly in order: the next console opens only after the previous one is done. The counters on the cards show what is still missing.

4.1. Password guessing

1. Keep the account `anna` and the address `203.0.113.77`.
2. Send five different wrong passwords. You can use the buttons `123456`, `qwerty`, `anna2026`, `sunflower`, `letmein` one after another.
3. From the intercepted note guess that the right password is `sunflower7`, and make the sixth attempt.

Important: a successful login made **before** five failures does not finish the mission. In this case the site explains the reason. Collect five failures and log in successfully again.

Compare the live telemetry with the original log:

```
grep '"event_type": "auth.login"' logs/events.jsonl | tail -n 2
```

Expected result: the card “Get access as anna” (“Получить доступ как anna”) is green, and the block of spraying is open. In the log you see `auth.login / failure` and, after the series, `auth.login / success`.

Question for the report (item 2): write down the values of `event_type`, `outcome` and `actor.id` from the record of a failed login.

4.2. Password spraying

Keep the password `123456` and the address `198.51.100.23`. Send attempts for the logins that the site suggests. After each attempt the field switches to the next login by itself. You need eight different accounts.

Expected result: the progress reaches `8/8`, the card of spraying becomes green, and the search through orders opens.

4.3. Looking for other people’s orders

Act as anna. Start with order 1001 and press **Open order** (“Открыть заказ”). After each request the number grows by itself. Orders 1001 and 1002 belong to anna. From 1003 you get refusals. Collect five refusals.

Expected result: after you check orders 1001–1007 the card becomes green, and the switching off of MFA opens.

4.4. Switching off MFA

Keep the user anna and leave the ticket field empty. Press **Disable MFA** (“Отключить MFA”).

Expected result: all four cards are green. The telemetry has 22 events: 13 login errors, 5 access refusals and one MFA switch-off. The other events are a successful login and the reading of two own orders.

Step 5. The log and the first detection rule

A detection rule is a condition on the records of a log. We write it in the Sigma format. We start with a simple rule on a single event: someone switched off the second authentication factor.

The commands below run from the folder with the practice materials. Go to the folder for rule checks and create a folder for the rules:

```
cd ../detect
mkdir -p rules
```

Create the file `rules/01_mfa.yml`:

```
title: Second factor switched off without a ticket
id: 00000000-0000-0000-0000-0000000000b1
name: mfa_disabled
logsource: {product: shop-app, service: auth}
detection:
    selection:
        event_type: auth.mfa.disable
        outcome: success
    filter_ticket:
        context.ticket_id|exists: true
    condition: selection and not filter_ticket
level: high
tags: [attack.credential-access, attack.t1556.006]
```

The rule fires on the event `auth.mfa.disable`, except when the switch-off has a ticket (the field `context.ticket_id`).

The last mission of step 4 already created the event of MFA switch-off. Apply the new rule to the log:

```
python3 detect.py --events ../app/logs/events.jsonl --rules rules
```

Expected result: the program prints `сигналов: 1` (“alerts: 1”) and a line with the title of your rule. The program prints its messages in Russian.

Step 6. The attack and its detection

We write rules for three attacks and test them on real activity.

Base rules

Correlation rules refer to simple rules. Create the file `rules/00_base.yml`:

```yaml
title: Failed login
id: 00000000-0000-0000-0000-0000000000a1
name: failed_login
logsource: {product: shop-app, service: auth}
detection:
    selection: {event_type: auth.login, outcome: failure}
    condition: selection
---
title: Successful login
id: 00000000-0000-0000-0000-0000000000a2
name: successful_login
logsource: {product: shop-app, service: auth}
detection:
    selection: {event_type: auth.login, outcome: success}
    condition: selection
```

Password guessing

A series of failed logins to one account. Create `rules/02_bruteforce.yml`:

```yaml
title: Password guessing for an account
id: 00000000-0000-0000-0000-0000000000b2
name: bruteforce_account
correlation:
    type: event_count          # count events
    rules: [failed_login]      # failed logins
    group-by: [actor.id]       # separately for each account
    timespan: 10m              # in 10 minutes
    condition: {gte: 5}        # 5 or more
level: medium
tags: [attack.credential-access, attack.t1110.001]
```

Password spraying

One address tries one password against many accounts. Create `rules/03_spraying.yml`:

```yaml
title: Password spraying from one address
id: 00000000-0000-0000-0000-0000000000b3
name: password_spraying
correlation:
```

```
    type: value_count          # count DIFFERENT values of a field
    rules: [failed_login]
    group-by: [source_ip]      # by the source address
    timespan: 30m
    condition: {field: actor.id, gte: 8}   # 8 or more different accounts
level: high
tags: [attack.credential-access, attack.t1110.003]
```

Successful login after many failures

This is a sign of successful guessing. Create `rules/04_sequence.yml`:

```
title: Successful login after many failed attempts
id: 00000000-0000-0000-0000-0000000000b4
correlation:
    type: temporal_ordered     # events in a given order
    rules: [failed_login_burst, successful_login]
    group-by: [actor.id]
    timespan: 15m
level: high
tags: [attack.initial-access, attack.t1078]
---
title: Three or more failed logins
id: 00000000-0000-0000-0000-0000000000b5
name: failed_login_burst
correlation:
    type: event_count
    rules: [failed_login]
    group-by: [actor.id]
    timespan: 10m
    condition: {gte: 3}
```

Prepare the control log

The manual quest showed how the attacks work, but its log depends on the number of your attempts. For a repeatable check press **Reset the lab** (“Сбросить стенд”) in the browser, and then run `simulate.py`. The script repeats the same four attacks and adds the normal work of users:

```
cd ../attack
python3 simulate.py --url http://localhost:8080
```

Apply all rules

```
cd ../detect
python3 detect.py --events ../app/logs/events.jsonl --rules rules --out
alerts.jsonl
```

Expected result: the program reports `Событий: 39` (“events: 39”) and `сигналов: 5` (“alerts: 5”). These rules fire: password guessing (the account `anna`), spraying (one address), switch-off of the second factor, and **two** alerts of the rule “successful login after many failures”, for `anna` and for `boris`. In the browser the same 39 events stay available for a visual check.

Question for the report (item 3): which rules fired, and for which accounts or addresses? For each alert, say if it is true or false.

Step 7. Tuning and the report

One alert of the rule “successful login after many failures” is false. The user `boris` made three mistakes in the password and logged in on the fourth attempt, because he forgot the password. This is not an attack.

Tune the rule. In the file `rules/04_sequence.yml`, raise the threshold of the series of failures from 3 to 5 (`condition: {gte: 5}` in the second document), and apply the rules again:

```
python3 detect.py --events ../app/logs/events.jsonl --rules rules
```

Expected result: the false alert for `boris` is gone. The guessing of the password for `anna` (7 failures) is still detected. There are 4 alerts in total.

Question for the report (item 4): what is the price of this change? Which attack can the rule now miss?

Fill in the report with the template `report-template.md` and give it to the teacher together with the folder `rules/`.

Extra task: catch an IDOR

The attacker went through other people's orders. Under the account `anna` the attacker asked for orders 1001–1009 and got access refusals (`authz.access` with the result `failure`). First find this series in the live telemetry of the browser. Then write a rule that detects the search through other people's objects: `event_count` of the event `authz.access` (failure), grouped by `actor.id`. How many access refusals are recorded for `anna`?

If something went wrong

Sign	What to do
`docker compose ps` does not show `Up`	Look at the log of the container: `docker compose logs`
`сигналов: 0` (“alerts: 0”) after the attack	Check that `rules/00_base.yml` exists. Correlation rules do not work without the base rules
An error in a rule that you do not understand	Check the indents in YAML (exactly 4 spaces per level) and run `python3 detect.py` again. The program names the file with the error